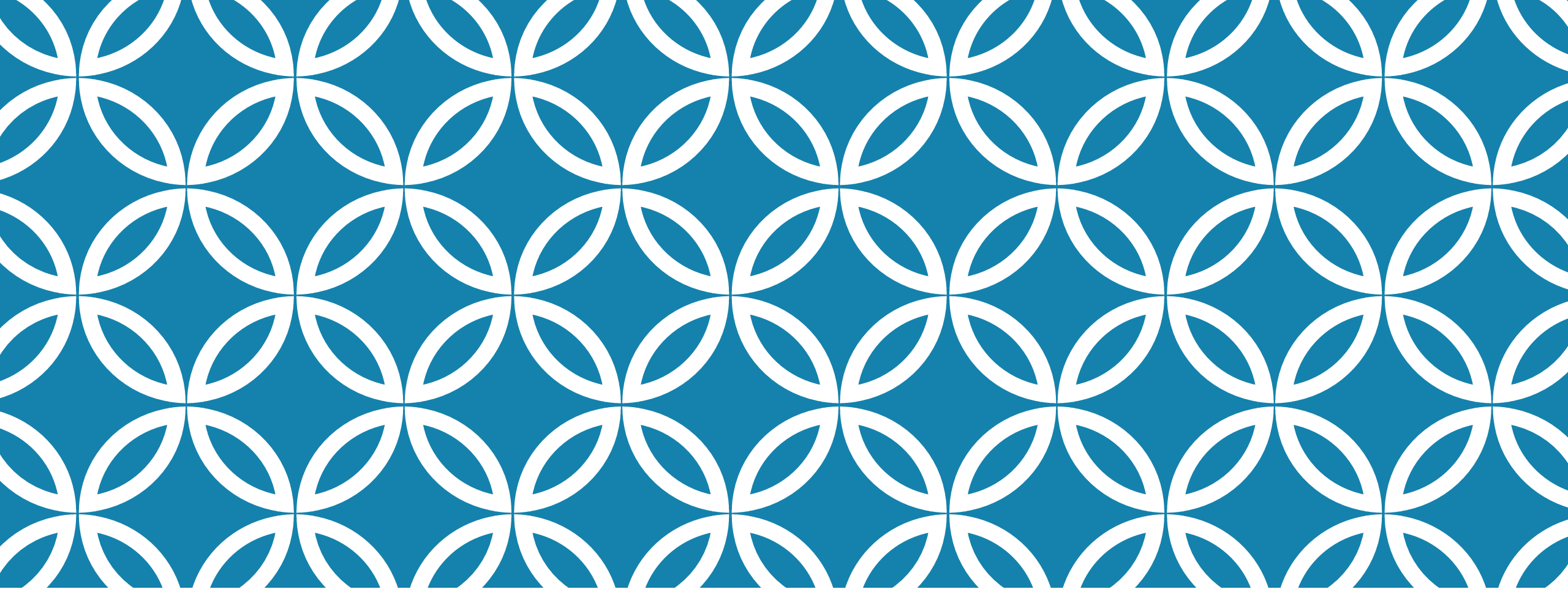




OBJECT ORIENTED PROGRAMMING

WEEK 4

12-17 SEP, 2022

Instructor:

Abdul Aziz
Assistant Professor
(School of Computing)
National University- FAST (KHI Campus)

ACKNOWLEDGMENT

- Publish material by Virtual University of Pakistan.
- Publish material by Deitel & Deitel.
- Publish material by Robert Lafore.



WEEK 4, DAY 1

Inline functions

INLINE FUNCTIONS

- What is the use of function
It is used to reduce the save the memory space when a function is likely to be called many times.
- Inline functions can be implemented using the keyword *inline*.
- The inline is a request to the compiler.
- The function always can not be inline by the compiler
- The complicated functions will not be inline.
- Just like a macro, but different from macro.

Disadvantages of using function

- Every time a function is called, it takes a lot of extra time in executing a series of instructions.
- In the following ways of execution it takes more time
 - 1. Jumping to the function
 - 2. Saving in Register.
 - 3. Pushing arguments into the stack.
 - 4. Returning to the calling function.

INLINE FUNCTION

- C++ provides an inline functions to reduce the function call overhead.
- Inline function is a function that is expanded in line when it is called.
- When the inline function is called whole code of the inline function gets inserted or substituted at the point of call.
- This substitution is performed by the C++ compiler at compile time.
- Inline function may increase efficiency if it is small.

The syntax for defining the function inline is:

```
inline return-type function-name(parameters) {
```

```
// function code
```

```
}
```

POINTS TO REMEMBER

Remember, inlining is only a request to the compiler, not a command.

Compiler can ignore the request for inlining.

Compiler may not perform inlining in such circumstances like:

- 1) If a function contains a loop. (for, while, do-while)
- 2) If a function contains static variables.
- 3) If a function is recursive.
- 4) If a function return type is other than void, and the return statement doesn't exist in function body.
- 5) If a function contains switch or goto statement.

INLINE FUNCTIONS ADVANTAGES

- 1) Function call overhead doesn't occur.
- 2) It also saves the overhead of push/pop variables on the stack when function is called.
- 3) It also saves overhead of a return call from a function.
- 4) When you inline a function, you may enable compiler to perform context specific optimization on the body of function. Such optimizations are not possible for normal function calls. Other optimizations can be obtained by considering the flows of calling context and the called context.
- 5) Inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function call preamble and return.

INLINE FUNCTION DISADVANTAGES

- 1) The added variables from the inlined function consumes additional registers, After in-lining function if variables number which are going to use register increases than they may create overhead on register variable resource utilization. This means that when inline function body is substituted at the point of function call, total number of variables used by the function also gets inserted. So the number of register going to be used for the variables will also get increased. So if after function inlining variable numbers increase drastically then it would surely cause an overhead on register utilization.
- 2) If you use too many inline functions then the size of the binary executable file will be large, because of the duplication of same code.
- 3) Too much inlining can also reduce your instruction cache hit rate, thus reducing the speed of instruction fetch from that of cache memory to that of primary memory.
- 4) Inline function may increase compile time overhead if someone changes the code inside the inline function then all the calling location has to be recompiled because compiler would require to replace all the code once again to reflect the changes, otherwise it will continue with old functionality.
- 5) Inline functions might cause thrashing because inlining might increase size of the binary executable file.

EXAMPLE

```
class S
{
public:
    inline int square(int s) // redundant use of inline
    {
        // this function is automatically inline
        // function body
    }
};
```

```
include <iostream>
using namespace std;
class operation
{
    int a,b,add,sub,mul;
    float div;
public:
    void get();
    void sum();
    void difference();
};
inline void operation :: get()
{
    cout << "Enter first value:";
    cin >> a;
    cout << "Enter second value:";
    cin >> b;
}

inline void operation :: sum()
{
    add = a+b;
    cout << "Addition of two numbers: " << a+b << "\n";
}

inline void operation :: difference()
{
    sub = a-b;
    cout << "Difference of two numbers: " << a-b << "\n";
}
```

WEEK 4, DAY 2 & 3

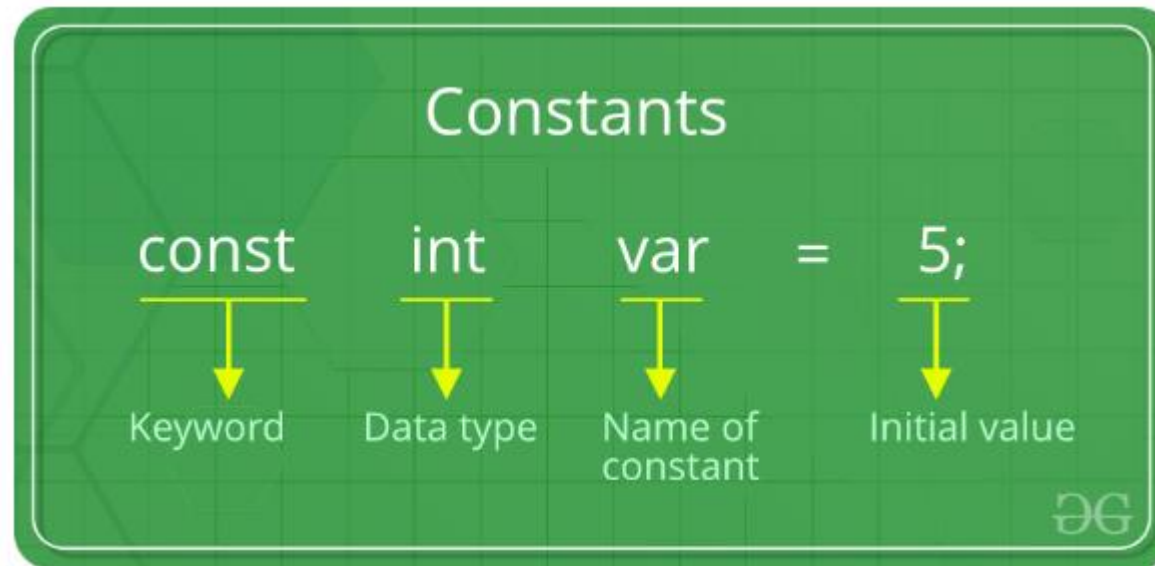
- Constant

- Data

- Functions

CONSTANT

- As the name suggests the name constants are given to such variables or values in C/C++ programming language which cannot be modified once they are defined.
- They are fixed values in a program.



DEFINING CONSTANTS

1. Using `#define` preprocessor directive
2. Using a `const` keyword

```
#define identifierName value
```

- **identifierName:** It is the name given to constant.
- **value:** This refers to any value assigned to identifierName.

USING A *CONST* KEYWORD

How to declare constants

`const int var;`



`const int var;
var = 5;`



`const int var = 5;`



EXAMPLE

```
#include <stdio.h>

int main()
{
    // int constant
    const int intVal = 10;

    // Real constant
    const float floatVal = 4.14;

    // char constant
    const char charVal = 'A';

    // string constant
    const char stringVal[10] = "ABC";
}
```


MEMBER INITIALIZER LIST

const data members must be initialized using Initializer List

```
#include<iostream>
using namespace std;

class Test {
    const int t;
public:
    Test(int t):t(t) {} //Initializer list must be used
    int getT() { return t; }
};

int main() {
    Test t1(10);
    cout<<t1.getT();
    return 0;
}
```

CONST MEMBER FUNCTIONS

- Constant member functions are those functions which are denied permission to change the values of the data members of their class.

<return_type> <function_name>() const

Example :

```
int get_data() const {  
    //function body  
}
```

DEFINITION OUTSIDE THE CLASS.

```
<return_type> <class_name> :: <function_name>() const {  
    //function body  
}
```

Example:

```
int Demo :: get_data() const { }
```

CONSTANT OBJECT

- Like member functions and member function arguments, the objects of a class can also be declared as **const**.
- an object declared as const cannot be modified (its state).
- A const object can be created by prefixing the const keyword to the object declaration.
- Any attempt to change the data member of const objects results in a compile-time error.